

Urban Flow Analytics

Technical Report — Upfront Pricing, Arrival Estimation, Demand Forecasting and Mobility Hotspots

Team Atreides

SLIIT Codefest Datathon 2026 · Round 1

48,601,782 taxi trips · April 2025 – March 2026

Live system: [Atreides-Datathon Deployed link](#)

Source code: [Atreides_CodefestDatathon2026 GitHub Repo](#)

Executive summary

Data. 48,601,782 trips processed in full by streaming each monthly CSV from the archive into DuckDB. 90.8% of rows are valid for price/time modelling and 94.3% are genuine pickups for demand.

Upfront fare. LightGBM on booking-time features only: MAE \$3.87, R^2 0.827 on an unseen future period (21.0% lower MAE than a corridor lookup).

Arrival time. MAE 3.96 min, R^2 0.804; 75.5% of trips within ± 5 minutes.

Demand. One global model forecasts hourly pickups 1–72 h ahead for the 10 busiest zones with WAPE 16.1% (R^2 0.898) vs 22.1% for the seasonal-naïve dispatch rule.

Hotspots. Six zone archetypes; morning movement is concentrated inside the Manhattan core, late-night movement shifts to the nightlife districts of the Lower East Side, East/West Village and Williamsburg.

Contents

1. Problem and data.....	3
2. Data quality assessment	3
3. Data preprocessing and feature engineering	5
4. Model development methodology.....	6
4.1 Upfront fare and arrival-time models.....	6
4.2 Demand forecasting (fleet dispatcher)	6
4.3 Hotspot and origin-destination flow clustering.....	6
5. Evaluation metrics and results	7
5.1 Why these metrics	7
5.2 Upfront fare	7
5.3 Arrival time	7
5.4 Did the preprocessing and features help? (ablation)	8
5.5 Demand forecasting	8
5.6 Hotspots and flows.....	9
6. Solution architecture	11
7. Key findings, business implications and conclusions	11
8. Bonus tracks	13
8.1 Track 6 — Business dashboard: “Flex Fare: is upfront pricing paying off?”	13
8.2 Track 5 — AI Mobility Assistant.....	13
8.3 Live deployment.....	14

1. Problem and data

The city's taxi operation needs four capabilities: a trustworthy **upfront price** locked before the ride, an accurate **arrival time**, a **demand forecast** that tells dispatchers where to position cabs 24–72 hours ahead, and a **big-picture view of travel patterns** for planners. All four rest on raw operational records that contain reversals, missing blocks and device faults, so data quality comes first.

Two datasets were provided: twelve monthly trip files (April 2025 – March 2026, 20 fields, **48,601,782 rows, 5.2 GB**) and a 265-row zone lookup (borough, zone name, service zone) joined on the pickup and drop-off location IDs.

Engineering constraint. The raw data is larger than the free disk on the development machine. Instead of extracting it, *src/ingest.py* streams each CSV out of the zip straight into DuckDB and in one pass (≈ 9 s per month) writes (i) anomaly counts over every row, (ii) hourly zone pickups, origin-destination flows and daily KPIs over every row, and (iii) a reproducible 5 % Bernoulli sample (2.43 M trips) for supervised learning. Data-quality percentages, demand series and flow maps therefore describe the full population; only the fare and duration models are trained on the sample, which is far beyond the size at which gradient-boosting accuracy plateaus.

2. Data quality assessment

Before deciding how to treat an anomaly we looked for its cause. Four root causes explain most of the problems:

- **Structural missing block = Flex Fare.** 25.5% of rows miss rider count, rate class, store-and-forward flag, congestion and airport fee together. Every one has settlement code 0 (Flex Fare, the app-based upfront-price product) — complete trips whose metering fields were not transmitted.
- **Negative fares are accounting reversals.** All 5.0% come from a single provider (Curb, code 2) and sit in Flex, Dispute and No-charge settlement codes: they refund an earlier charge rather than describe a trip.
- **One provider never records drop-off time.** For provider 7 (Helix) dropoff = pickup on 100 % of rows (1.3% of the data) while its fares and distances are valid.
- **Odometer failures.** 2.6% of trips report 0 miles but were charged (median $\approx$ \$20, $\approx$ 13 min) — real rides with a broken distance reading.

Anomaly	Rows	% of all rows	Decision	Justification
Negative fare / total	2,405,603	4.950%	Drop	Reversal entries, not trips
Zero fare	24,577	0.051%	Drop	Cancelled / no-charge; no price signal
0 miles but charged	1,267,110	2.607%	Drop for fare/ETA · keep for demand	Distance broken, pickup real
Rider count = 0	231,578	0.476%	Impute 1 + flag	A charged trip carried ≥ 1 rider
Missing block (Flex Fare)	12,405,268	25.524%	Impute + is_flex flag	Real trips; dropping biases all models
Drop-off before pickup	1,942	0.004%	Drop	Physically impossible
Meter time < 1 min	1,190,046	2.449%	Drop (except Helix)	False starts / voids
Helix zero duration	642,536	1.322%	Keep; duration $\rightarrow$ missing	Device defect; fare valid
Duration > 3 h	18,613	0.038%	Drop	Meter left running
Speed > 80 mph	13,589	0.028%	Drop	Distance or clock corrupt
Distance > 100 mi	2,817	0.006%	Drop	Odometer glitch (up to 10^5 mi)
Base fare > \$500	860	0.002%	Drop	Keying error
Pickup outside file month	170	0.000%	Drop	Corrupt clock (e.g. 2008)
Unknown zone 264/265	360,881	0.743%	Keep as category	Valid trips, unknown geography
Rate class 99 (unknown)	1,030,466	2.120%	Keep as category	Separate level for the model

Table 1 — Anomaly audit over all 48,601,782 rows (a row can carry several anomalies).

Decision policy. A row is *dropped* only when it does not describe a real, correctly measured trip — imputing a price for a refund would invent data. A field is *imputed and flagged* when the trip is real and only a low-information attribute is missing, so the model can still learn any residual difference. A row is *filtered per task* when a single field is broken: Helix trips stay in the fare and demand data but leave the duration target. **Net effect: 90.8% of rows are used for fare/duration modelling and 94.3% count as pickups for demand.**

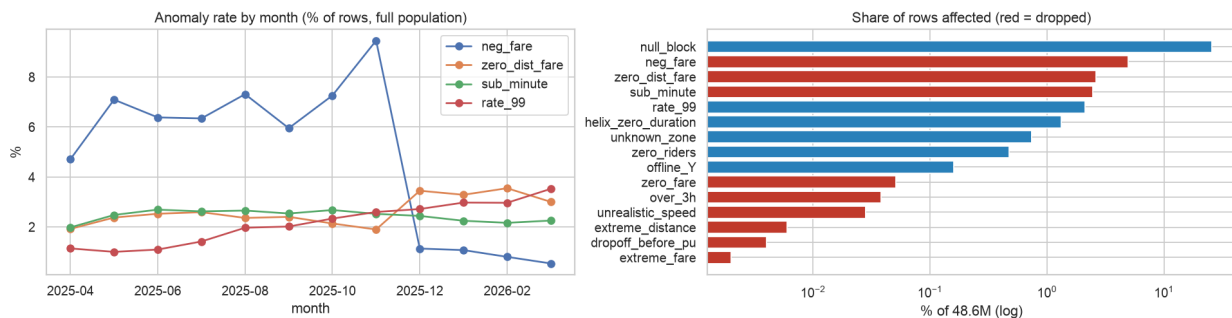


Figure 1 — Monthly anomaly rates (left) and share of rows affected per type (right; red = dropped).

Drift. Reversals fall from 6–9 % of rows (Apr–Nov 2025) to ≈ 1 % from December 2025, and the unknown rate class triples over the year. At the same time Flex Fare prices rose from $\approx$ \$5 to $\approx$ \$7.5 per mile while the standard meter tariff stayed constant — a reporting and pricing regime change that the models must survive.

3. Data preprocessing and feature engineering

Leakage rule. Fare and arrival time are predicted at booking time, so no feature may use anything measured after the meter starts: metered distance, drop-off time, tips, tolls, surcharges and payment method are excluded.

Validation is strictly temporal — **train Apr–Dec 2025, validation Jan 2026, test Feb–Mar 2026** — which mirrors deployment and avoids the optimism of random splits.

Group	Features	Rationale
Time	hour, minute of day, day of week, weekend, US holiday, sin/cos hour; t_index (days since start, fare model only)	Traffic and demand follow hour-of-week; t_index lets trees carry the Dec-2025 Flex repricing forward
Location	origin and destination zone (LightGBM categorical), boroughs, airport flags (JFK, LGA, EWR), same-zone, unknown-zone	Zones encode geography that the data lacks as coordinates
Corridor history	median distance and duration of the origin→destination pair, median duration for the pair in the same 3-h band, pair frequency	Pre-trip stand-in for a routing engine; learned on train only with back-off to zone and global medians for unseen pairs
Booking context	rider count, provider, rate class, Flex-Fare flag	Known at booking: airport flat and negotiated rates are agreed up front

Table 2 — Booking-time feature set (24 features).

Categoricals and missing values. Zone, borough, provider and rate class are native LightGBM categoricals (with smoothing), avoiding a 260-column one-hot. Missing rider count and rate class are imputed (1 and 99) with explicit flags; unseen zones at inference map to LightGBM's missing branch. Timestamps become cyclic and calendar features; calendar month is deliberately excluded because test months never occur in training.

Demand features. For each (zone, forecast origin, horizon 1–72 h) row: pickups at the same hour of week 1–4 weeks earlier (last value, mean, median), same hour on the latest observed day, last-24 h and last-168 h means, last observation, a recent-level ratio, calendar/holiday flags, zone and horizon — all strictly before the forecast origin.

Clustering features. Per zone: share of pickups and of drop-offs in five time bands (06–10, 10–16, 16–20, 20–24, 00–06), pickup/drop-off balance, log volume and weekend share; standardised before K-Means.

4. Model development methodology

4.1 Upfront fare and arrival-time models

Both targets use the same ladder of increasingly capable models, evaluated on the same untouched test period:

- **Global median** — the floor any model must beat.
- **Corridor median lookup** — what a simple price table per origin-destination pair would do.
- **Ridge regression** on numeric features and one-hot rate class — a strong linear benchmark.
- **LightGBM (default)** and **LightGBM (tuned)** — gradient-boosted trees handle the non-linear interactions (zone $\times$ hour $\times$ rate class), native categorical and 1.7 M training rows in about a minute.

Hyper-parameter tuning. Random search over `num_leaves`, `min_child_samples`, learning rate, column subsampling, L2 regularisation and categorical smoothing (12 configurations on a 300 k-row training subsample, scored by validation RMSE, early stopping on the validation month). The final model is whichever LightGBM wins on validation; test results are reported only once. Selected: fare — LightGBM (tuned); duration — LightGBM (tuned).

Deployment wrapper. Each saved model (.pkl) is a TripModel object holding the booster, the train-only corridor statistics, the zone lookup and the category levels, so a raw booking request (time, zones, riders, provider, rate class, Flex flag) can be scored directly.

4.2 Demand forecasting (fleet dispatcher)

Hourly pickups for the 10 busiest zones (Upper East Side South, Midtown Center, JFK Airport, Upper East Side North, Penn Station/Madison Sq West, Midtown East, Times Sq/Theatre District, Lincoln Square East, LaGuardia Airport, Murray Hill; $\approx 34\%$ of all trips) were built from all genuine pickups. Every day at 23:00 a forecast is issued for the next 72 hours. We use a **single global, direct multi-horizon LightGBM** with a Poisson objective (counts): one model serves every zone and horizon, shares strength across zones, and needs no recursive feeding of its own errors. Baselines are the standard dispatcher heuristics — *same hour last week* and *4-week hour-of-week average*. Eight tuning configurations were scored on validation WAPE. Splits: training origins 28 Apr 2025 – 28 Jan 2026, validation 29 Jan – 25 Feb, test 26 Feb – 28 Mar 2026 (forecasts cover through 31 Mar).

4.3 Hotspot and origin-destination flow clustering

Zones with $\geq 2,000$ trips were clustered with K-Means on their temporal activity profiles (Section 3). Silhouette scores fall from 0.283 ($k = 3$) to 0.203 ($k = 6$); we chose $k = 6$ because it is the smallest k that separates nightlife, airports and the two residential types — distinctions the business needs and that $k = 3$ merges. Trips were then aggregated to archetype $\rightarrow$ archetype flows per time band, turning ≈ 35 k zone pairs into a readable map of city movement.

5. Evaluation metrics and results

5.1 Why these metrics

Metric	Used for	Why it fits the challenge
MAE	fare (\$), duration (min), demand	Same unit as the target: the average "surprise" a rider or dispatcher experiences; robust to the heavy tails that survive cleaning. Primary comparison metric.
RMSE	all regressions; tuning objective	Punishes the rare large miss — the case that destroys trust in a locked-in price or ETA. RMSE $\gg$ MAE signals a heavy error tail.
R ²	all regressions	Scale-free variance explained; compares fare, duration and demand models and baselines on one axis.
Median AE, % within ± 5 / ± 2 min / ± 5 min	fare, duration	Business-facing view of the typical rider experience and the share of on-promise predictions.
WAPE	demand	Total absolute error $\div$ total demand. Stable when some zone-hours are small (MAPE explodes) and reads as "% of fleet mis-allocated".

Table 3 — Metric rationale. All models are compared on the same future test period, so improvements cannot come from leakage.

5.2 Upfront fare

Model	MAE (\$)	RMSE (\$)	R ²	Median AE (\$)
Global median	11.29	18.45	-0.142	6.30
Corridor median lookup	4.90	9.44	0.701	2.79
Ridge regression	5.11	9.04	0.725	3.21
LightGBM (default)	3.87	7.15	0.828	2.35
LightGBM (tuned)	3.87	7.17	0.827	2.35

Table 4 — Fare models on the Feb–Mar 2026 test set (351,718 trips). Highlighted: selected model.

Segment	Trips	MAE (\$)	RMSE (\$)	Within \$2	Within \$5
Flex Fare	89,156	6.42	10.12	26.6%	58.1%
JFK flat	7,742	0.81	1.37	95.0%	98.6%
Other rates	14,910	7.05	17.30	36.4%	67.8%
Standard meter	239,910	2.83	4.32	49.5%	85.5%

Table 5 — Fare error by pricing segment.

Standard-meter trips — two thirds of volume — are priced with MAE \$2.83 and 85.5% within \$5; JFK flat fares are almost exact. Most residual error sits in **Flex Fare** trips, whose price is set by the provider's own algorithm: even after the trip, distance and time explain only ≈ 50 % of their variance versus ≈ 99 % for metered trips. A *ceiling experiment* that adds the metered distance (not available before the trip) reaches MAE \$3.07 and R² 0.885, which quantifies the value of integrating a routing engine's distance estimate.

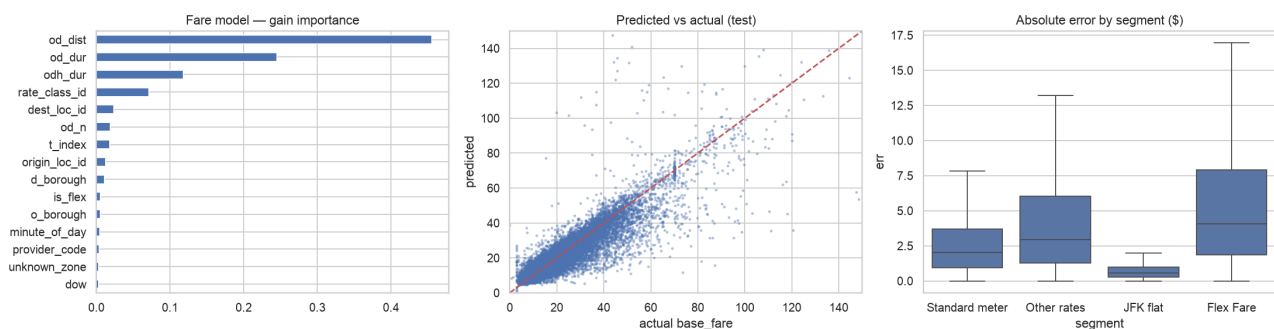


Figure 2 — Fare model: feature importance (gain), predicted vs actual, and error by segment.

5.3 Arrival time

Model	MAE (min)	RMSE (min)	R ²	Median AE (min)
Global median	9.19	14.67	-0.061	6.27
Corridor median lookup	5.03	8.84	0.615	3.05

Model	MAE (min)	RMSE (min)	R ²	Median AE (min)
Ridge regression	4.73	7.92	0.690	3.11
LightGBM (default)	3.98	6.33	0.803	2.70
LightGBM (tuned)	3.96	6.31	0.804	2.68

Table 6 — Duration models on the test set (Helix rows excluded from the target).

The selected model is within ± 2 minutes for 38.9% and within ± 5 minutes for 75.5% of trips; typical error grows with trip length (MAE ≈ 2.6 min for trips under 10 min, ≈ 4.6 min for 20–30 min trips), i.e. roughly 20 % of the ride for trips over 10 minutes. With metered distance the ceiling is MAE 3.35 min (R^2 0.844): the remaining gap is route choice and live traffic, which real-time traffic feeds would close.

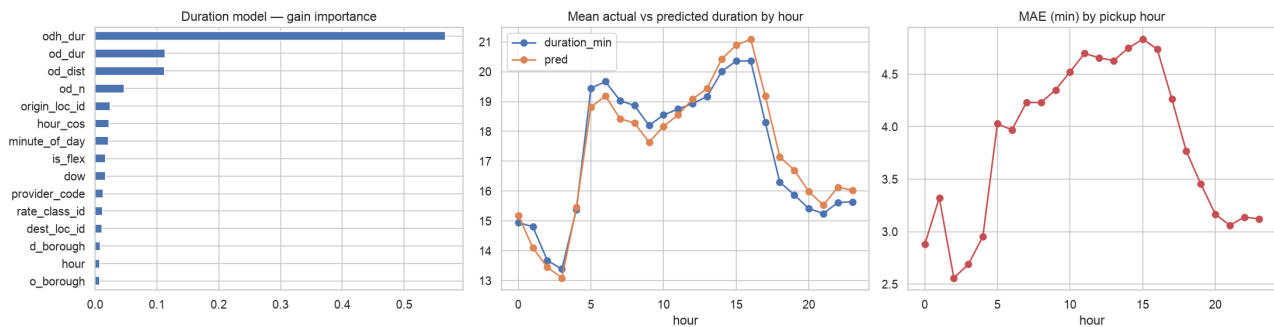


Figure 3 — Duration model: feature importance, hourly calibration and hourly error.

5.4 Did the preprocessing and features help? (ablation)

Fare model variant	MAE (\$)	RMSE (\$)	R ²
Full pipeline (final features)	3.87	7.15	0.828
– corridor statistics	3.92	7.18	0.827
– zone IDs (boroughs only)	3.93	7.35	0.819
$\pm t$ index (toggled)	4.14	7.85	0.793
– row cleaning (train on raw rows)	4.09	7.56	0.808

Duration model variant	MAE (min)	RMSE (min)	R ²
Full pipeline (final features)	3.98	6.33	0.803
– corridor statistics	3.98	6.28	0.806
– zone IDs (boroughs only)	4.07	6.53	0.790
$\pm t$ index (toggled)	4.16	6.69	0.780
– row cleaning (train on raw rows)	4.32	7.11	0.751

Table 7 — Ablations (default LightGBM, same split, same clean test set). One ingredient removed per row.

Row cleaning is the largest gain: training on raw rows costs 0.020 R^2 on fare and 0.052 on duration. **The time index is target-specific:** removing it costs the fare model 0.035 R^2 (it carries the Flex repricing forward), while adding it to the duration model costs 0.023 — so the two final models use different feature sets. **Zone identity** adds ≈ 0.01 R^2 to both. **Corridor statistics** add little once zone IDs are present, because the trees learn origin $\times$ destination interactions from the IDs; they are kept as the back-off for rare or unseen corridors and as interpretable inputs (explicit expected distance and time), at no accuracy cost.

5.5 Demand forecasting

Model	MAE	RMSE	WAPE	R ²
Seasonal naive (same hour last week)	38.6	64.8	22.1%	0.784
4-week hour-of-week mean	31.4	50.0	18.0%	0.871
LightGBM global (tuned)	28.1	44.5	16.1%	0.898

Table 8 — Hourly pickups per zone, 1–72 h ahead, test forecasts issued 26 Feb – 28 Mar 2026.

Horizon	MAE	RMSE	WAPE	R ²
1-24h	27.4	43.6	15.7%	0.902
25-48h	28.4	45.0	16.3%	0.895
49-72h	28.6	44.8	16.4%	0.897

Table 9 — Accuracy barely degrades from day 1 to day 3.

The global model cuts WAPE from 22.1% (seasonal naive) and 18.0% (4-week average) to 16.1%, and wins in every one of the ten zones. It is most accurate in the dense Manhattan zones (WAPE $\approx$ 13–15 %) and hardest at LaGuardia ($\approx$ 26%), where flight schedules and weather — not available in this data — drive demand.

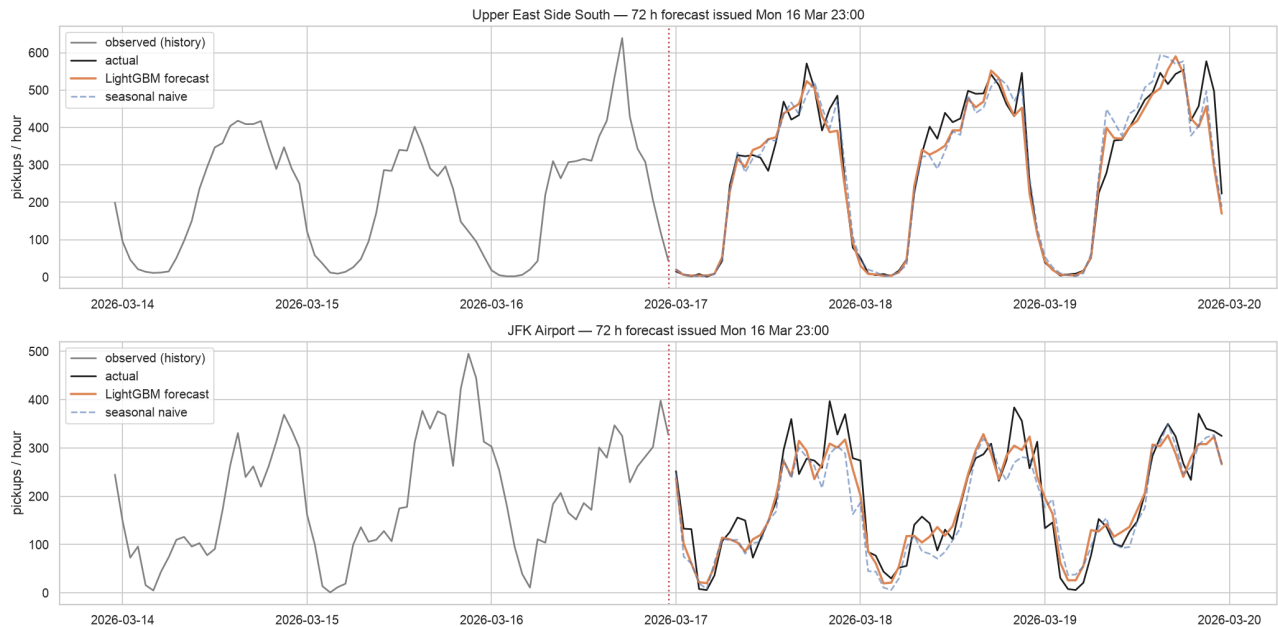


Figure 4 — 72-hour forecasts for Upper East Side South and JFK Airport against actuals and the seasonal-naive rule.

5.6 Hotspots and flows

Archetype	Zones	Examples	Morning share	Late-night share	Weekend share
Manhattan core (business & transit hubs)	41	Upper East Side South, Upper East Side North, Midtown Center, Times Sq/Theatre District	13%	6%	27%
Mixed-use Manhattan (dining & residential)	25	Clinton East, West Village, Gramercy, Flatiron	14%	14%	35%
Nightlife districts	14	East Village, Lower East Side, Alphabet City, Williamsburg (North Side)	12%	33%	50%
Outer-borough residential (morning origins)	75	East New York, Prospect-Lefferts Gardens, Flatbush/Ditmas Park, Canarsie	32%	12%	28%
Peripheral / low-volume & airports	21	LaGuardia Airport, East Harlem South, Newark Airport, Melrose South	20%	7%	26%
Uptown & Queens residential	58	Central Harlem North, Washington Heights South, Hamilton Heights, Astoria	22%	14%	33%

Table 10 — Zone archetypes (share of each zone's pickups in the band).

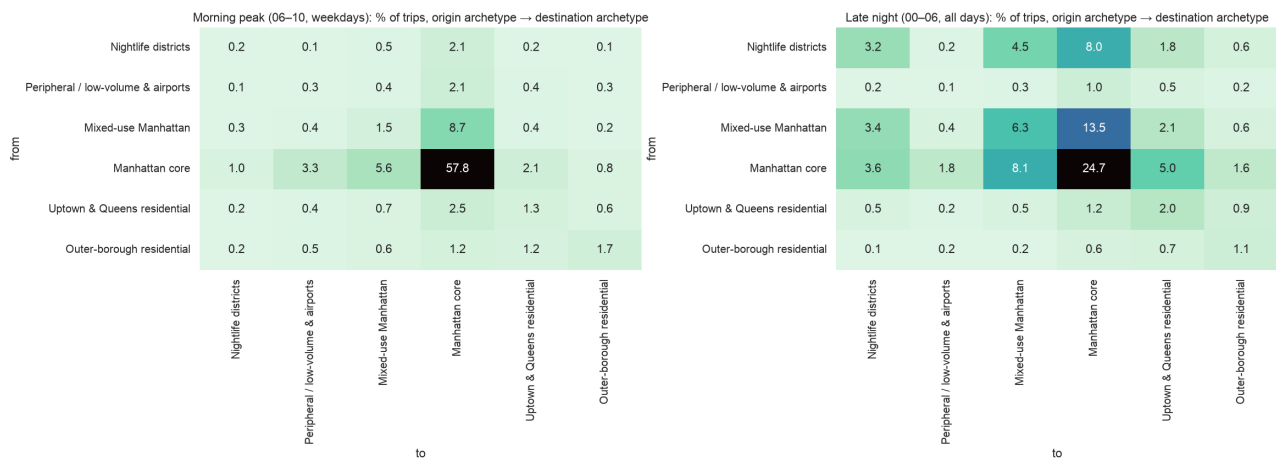


Figure 5 — Share of trips between archetypes: weekday morning peak vs late night.

Morning: about 58 % of weekday-morning trips stay inside the Manhattan core; top corridors are Upper East Side North → Midtown Center; Upper East Side North → Upper East Side South; Upper East Side South → Upper East Side North; Upper East Side South → Midtown Center. **Late night:** nightlife and mixed-use zones originate about 45 % of trips and the top corridors become short hops between them — West Village → East Village; East Village → Murray Hill; Lower East Side → East Village; East Village → Gramercy. Airports remain hotspots in every band.

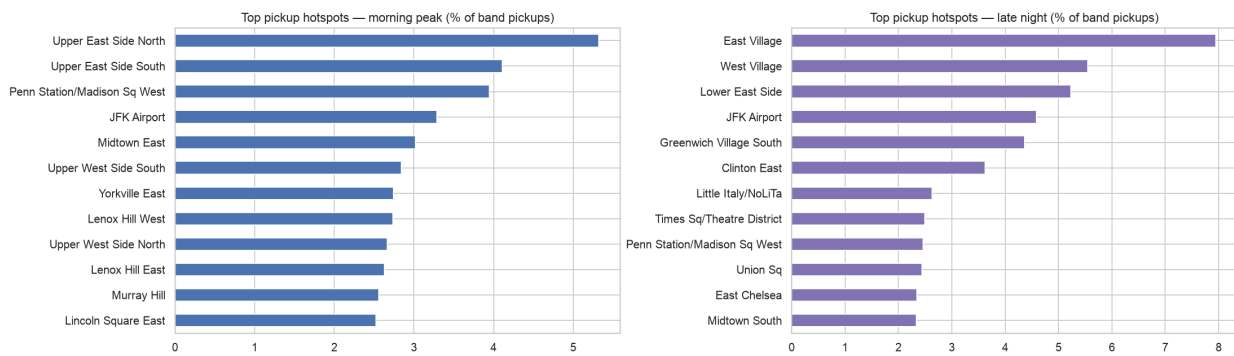


Figure 6 — Top pickup hotspots, morning peak vs late night.

6. Solution architecture

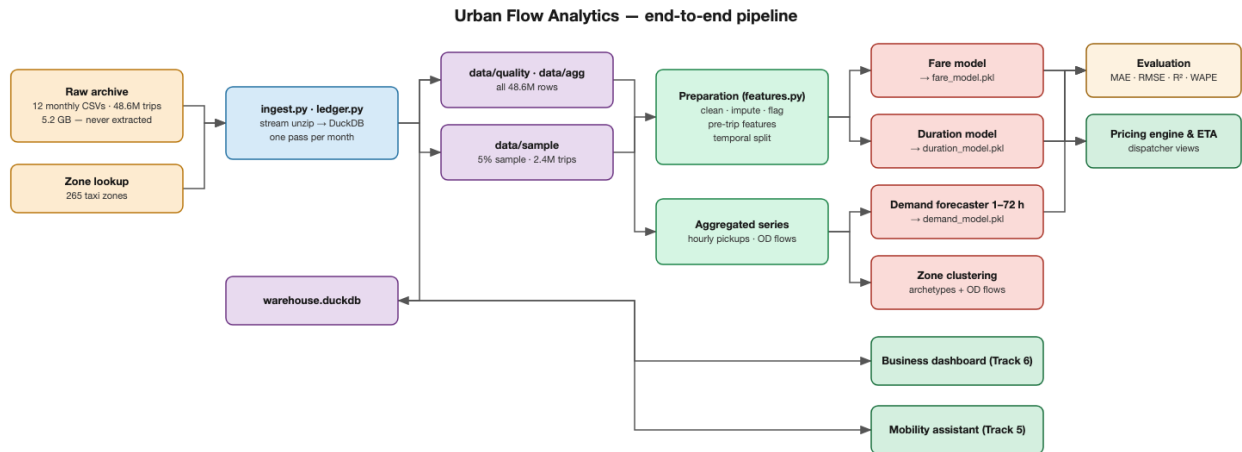


Figure 7 — End-to-end pipeline from raw archive to predictions.

- **Ingestion:** unzip → DuckDB stream per month; full-population audit and aggregates; 5 % sample.
- **Preparation:** one shared cleaning module (src/features.py) used by training and inference; temporal splits saved to data/splits.
- **Models:** fare_model.pkl and duration_model.pkl (booking request → price / ETA), demand_model.pkl (zone × 72 h forecast), zone archetype table for planners.
- **Operation:** retrain monthly (pricing drift observed in Dec 2025), monitor MAE per segment and WAPE per zone, alert when drift exceeds the validation range.

7. Key findings, business implications and conclusions

- **Data quality is a business issue.** About 9 % of records cannot support pricing; one provider emits all reversals and another never records drop-off time. Provider-level data SLAs would recover this data and make revenue reporting trustworthy.
- **Upfront pricing is ready for metered trips.** With MAE \$2.83 on standard-meter rides, prices can be locked with a small buffer. Flex Fare pricing is volatile (Dec-2025 regime change) and should be monitored separately with monthly retraining.
- **Route knowledge is the biggest remaining lever.** A routing engine's distance estimate would move fare R^2 from 0.83 towards 0.89 and cut ETA error by about 15.5%.
- **Demand is predictable three days ahead.** The global forecaster beats the dispatcher rules at every horizon and in every top zone, enough to plan shifts and pre-position cabs a day to three in advance.
- **The city has two rhythms.** Reposition idle cabs from Midtown to the nightlife cluster after 22:00 and back towards residential and core zones before 07:00; keep a standing airport allocation. Combined with the forecast this reduces empty cruising and passenger wait times.

Limitations and next steps. No weather, events or flight data were available — adding them would help the airport zones most. Models were trained on a 5 % sample for speed; the pipeline scales to the full data unchanged. Quantile versions of the fare model (P10/P90) would let the pricing engine choose its buffer explicitly.

8. Bonus tracks

8.1 Track 6 — Business dashboard: “Flex Fare: is upfront pricing paying off?”

Problem. Flex Fare (app-booked, price agreed before pickup) grew from 16 % to 29 % of trips and was repriced in December 2025. Management needs to know whether it is paying off — for the company, riders and drivers.

What the data says. Since the repricing a Flex trip bills \$33.24 versus \$29.93 for a metered card trip (it was the cheapest product before), and Flex costs \$7.34 per mile against \$5.74 on the meter. Yet only 1.6 % of Flex fare comes back to drivers as a recorded tip, against 21 % on card trips, and Flex share fell from 29 % to 23 % in March 2026. Refund reversals — mostly Flex — fell from about 250–400 k to under 50 k records a month when the new pricing launched.

Recommendations. (1) add a tip step to the Flex checkout — about \$28 M a year more for drivers if Flex tips reach half the card rate; (2) cap Flex quotes at a margin above the Section 5.2 fare model's “fair meter estimate”; (3) monitor Flex share and per-mile premium monthly with retraining; (4) keep the new refund process; (5) shift driver incentives to 18:00–02:00, when a meter-hour earns most.

The dashboard is one offline HTML file for management: filters for borough, months and day type recompute every chart, KPI and narrative sentence; a scenario slider sizes the tip opportunity; every chart has a data table; small selections trigger a reliability warning; views can be shared by link.

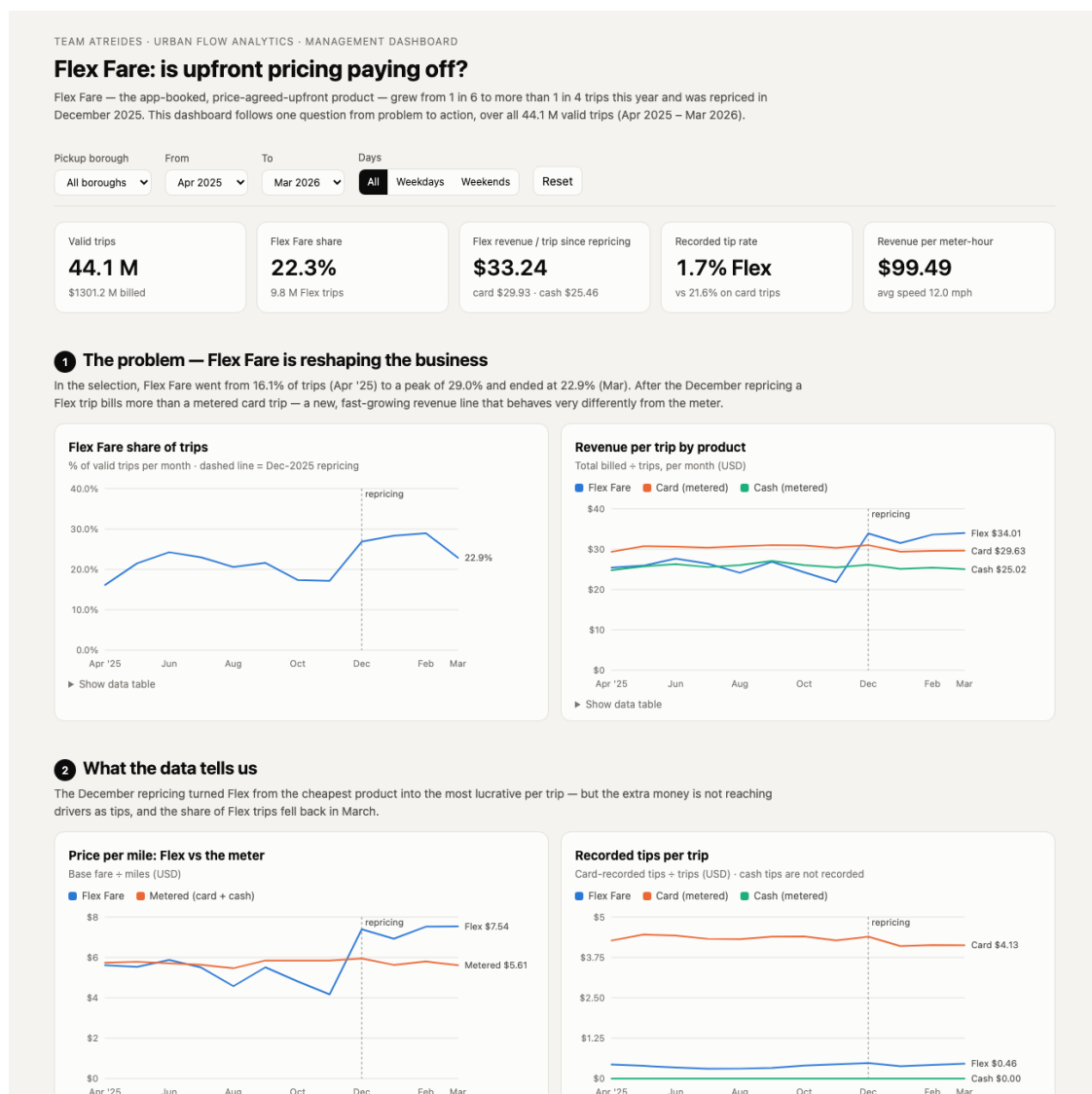


Figure 8 — Management dashboard (top of page).

8.2 Track 5 — AI Mobility Assistant

Plain-English questions are answered from a read-only DuckDB warehouse built from all 48.6 M records. **Claude engine:** Claude Opus 5 with two tools — `find_zones` (resolves “JFK”, “midtown”) and `run_sql` — plans a query, reads the result, retries on errors, keeps the conversation for follow-ups, asks one clarifying question when the answer would materially depend on it, and declines out-of-scope questions (weather, predictions, dates outside the data). **Offline engine:** a transparent rule-based parser for the common question families, used automatically without an API key or if the API fails.

Safety by design. Four independent layers: the database is opened read-only with external file/network access disabled and configuration locked; DuckDB's own parser must classify the query as a single `SELECT`; a deny-list blocks `file`, `extension` and `settings` functions; results are capped at 200 rows and queries time out after 15 s. Tests confirm `DROP`, multi-statement, `COPY`, `ATTACH`, `PRAGMA`, file reads and runaway joins are all stopped, and that the engine itself refuses file access even if the guard were bypassed.

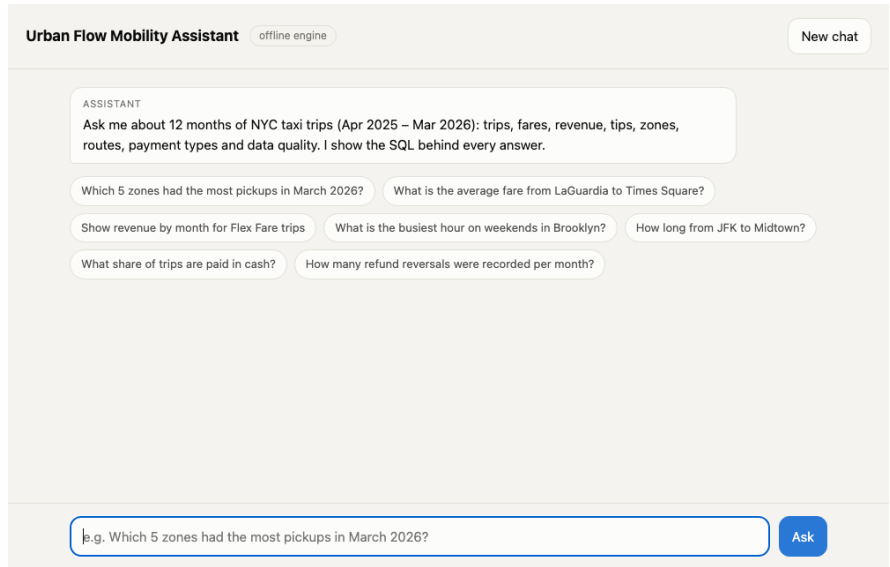


Figure 9 — Assistant web chat with suggested questions; each answer shows its SQL and result table.

8.3 Live deployment

Both bonus tracks run as a hosted service, not just as local files. A small Ubuntu 24.04 VPS serves the dashboard as a static page through nginx, while the assistant runs as a hardened systemd service (unprivileged user, read-only view of its own directory) bound to localhost behind the proxy. Every visitor gets an isolated conversation; the API is rate-limited per session and per IP.

What	Address
Business dashboard (Track 6)	atreides-datathon.duckdns.org
Mobility assistant (Track 5)	atreides-datathon.duckdns.org/assistant
Source code and full history	github.com/CJChANu/Atreides_CodefestDatathon2026

Live: <http://atreides-datathon.duckdns.org>

Source: https://github.com/CJChANu/Atreides_CodefestDatathon2026

Engine and cost. The assistant is provider-agnostic: it uses Claude when an Anthropic key is present, any OpenAI-compatible endpoint when one is configured (it currently runs on a free OpenRouter model), and otherwise its own offline rule-based engine. A model that answers with figures without querying the warehouse is rejected and the offline engine responds instead, so no number reaches the reader without SQL behind it.